

Ex.No:5A

Implement the Digital Signature Algorithm (DSA)

Date:

AIM:

ALGORITHM:

CODE:

```
import hashlib

def modular_inverse(number, modulus):
    return pow(number, -1, modulus)

def message_hash(message, q):
    hash_value = hashlib.sha256(
        message.encode("utf-8")
    ).digest()
    hash_integer = int.from_bytes(
        hash_value,
        byteorder="big"
    )
    return hash_integer % q

def generate_signature(message, p, q, g, private_key, k):
    hash_value = message_hash(message, q)
    r = pow(g, k, p) % q
    k_inverse = modular_inverse(k, q)
    s = (
        k_inverse
        * (hash_value + private_key * r)
    ) % q
    return r, s, hash_value

def verify_signature(message, signature, p, q, g, public_key):
    r, s = signature
    if not (0 < r < q and 0 < s < q):
        return False, None, None, None
    hash_value = message_hash(message, q)
    w = modular_inverse(s, q)
    u1 = (hash_value * w) % q
    u2 = (r * w) % q
```

```

    v = (
        (
            pow(g, u1, p)
            * pow(public_key, u2, p)
        ) % p
    ) % q
    return v == r, v, u1, u2
def main():
    p = 23
    q = 11
    g = 4
    private_key = 3
    public_key = pow(g, private_key, p)
    k = 7
    message = input("Enter the message: ")
    signature_r, signature_s, hash_value = generate_signature(
        message,
        p,
        q,
        g,
        private_key,
        k
    )
    print("\n----- DSA Parameters -----")
    print("Prime Number (p) :", p)
    print("Prime Divisor (q) :", q)
    print("Generator (g) :", g)
    print("\n----- Key Generation -----")
    print("Private Key (x) :", private_key)
    print("Public Key (y) :", public_key)
    print("\n----- Signature Generation -----")
    print("Message :", message)
    print("Message Hash :", hash_value)
    print("Random Number (k) :", k)
    print("Signature r :", signature_r)
    print("Signature s :", signature_s)
    print("Digital Signature :", (signature_r, signature_s))
    valid, v, u1, u2 = verify_signature(
        message,
        (signature_r, signature_s),
        p,
        q,
        g,
        public_key
    )

```

```

    )
    print("\n----- Signature Verification -----")
    print("u1 :", u1)
    print("u2 :", u2)
    print("v :", v)
    print("r :", signature_r)
    if valid:
        print("\nSignature Status : VALID")
    else:
        print("\nSignature Status : INVALID")
if __name__ == "__main__":
    main()

```

OUTPUT:

```

----- DSA Parameters -----
Prime Number (p) : 23
Prime Divisor (q) : 11
Generator (g) : 4

----- Key Generation -----
Private Key (x) : 3
Public Key (y) : 18

----- Signature Generation -----
Message : HELLO
Message Hash : 7
Random Number (k) : 7
Signature r : 8
Signature s : 6
Digital Signature : (8, 6)

----- Signature Verification -----
u1 : 3
u2 : 5
v : 8
r : 8

Signature Status : VALID

```

RESULT:

Ex.No:5B

Implement the Digital Signature Algorithm (DSA)

Date:

AIM:

ALGORITHM:

CODE:

```
import hashlib
p = 23
q = 11
g = 4
x = 3
y = pow(g, x, p)
k = 7
msg = input("Enter Message: ")
h = int(hashlib.sha256(msg.encode()).hexdigest(), 16) % q
r = pow(g, k, p) % q
k_inv = pow(k, -1, q)
s = (k_inv * (h + x * r)) % q
print("\nDigital Signature")
print("r =", r)
print("s =", s)
w = pow(s, -1, q)
u1 = (h * w) % q
u2 = (r * w) % q
v = ((pow(g, u1, p) * pow(y, u2, p)) % p) % q
print("\nVerification")
print("v =", v)
if v == r:
    print("Signature is Valid")
else:
    print("Signature is Invalid")
```

OUTPUT:

```
Enter Message: HELLO
```

```
Digital Signature
```

```
r = 8
```

```
s = 6
```

```
Verification
```

```
v = 8
```

```
Signature is Valid
```

RESULT: